Algorithms and Data Structures

18.09.2024

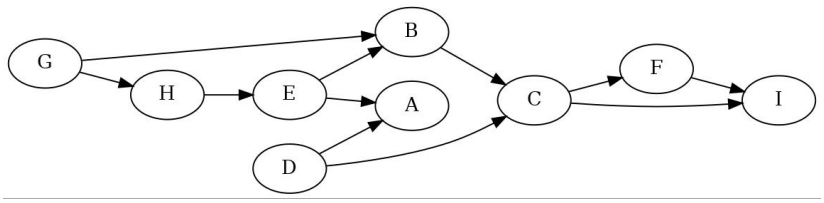
Exercise 1 (1.5 points)
The following expression is given in in-fix notation. Using its representation as a binary tree, convert it into pre-fix (Polish) and post-fix (Reverse Polish) notation.
`( ( A - B ) / ( C + ( D + E ) ) ) - ( ( F * ( G + H ) ) + ( I / L ) ) )`
Report the expression in pre-fix notation and post-fix notation. Notice that this is an open question that will be manually evaluated.

Exercise 2 (1.5 points)
Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to `M=19`. Insertions occur character by character using open addressing with quadratic probing (with $c_1 = 1$ and $c_2 = 1$). Each character is identified by its index in the English alphabet (i.e., `A=1`, ..., `Z=26`). Equal letters are identified by a different subscript (i.e., `A` and `A` become `A1` and `A2`).
`P A M P A M`
Indicate in which elements the last three letters of the sequence are placed, i.e., `P`, `A`, and `M`, in this order. Please report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response format: 3 4 11

Exercise 3 (1.5 points)
Given the following sequence of integers stored in an array, turn it in a heap, assuming to use an array as an underlying data structure. Assume that, in the end, the largest value is stored at the heap's root. Then, execute the first two steps of heapsort on the heap built at the previous step.
`15 5 6 7 8 10 3 2 11`
Report the final content of the entire array at the end of the above process. Please show the entire content of the array as a sequence of integer values separated by a single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8 etc.

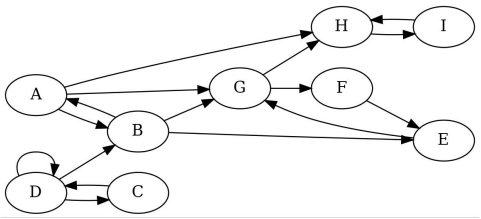
Exercise 4 (4.0 points)
Define the data structure heap as adopted in computer science. Clarify in which problems it is usually used. Report (in C code) the implementation of the functions heap-sort, heap-build, and heapify working on an array of real values.

Exercise 5 (1.5 points)
Visit the following graph in depth-first, starting at node G. Label each edge as tree (T), back (B), forward (F), and cross (C). When necessary, consider nodes and edges in alphabetic order.



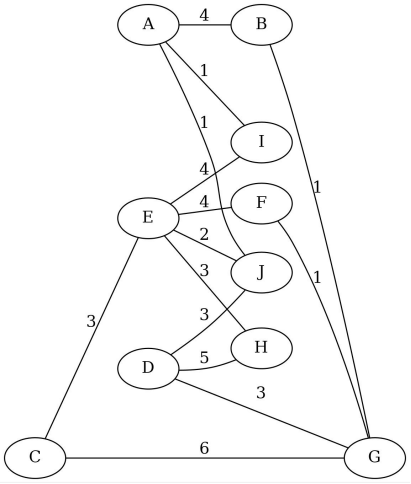
Report the labels of the edges CI, EA, and DA in this order. Please indicate the edge type of these 3 arcs with a single letter, i.e., T, F, B, C, separated by single spaces. No other symbols must be included in the response. The following is an example of the response: B T C

Exercise 6 (1.5 points)
Given the following directed graph, represent the reverse graph and find all strongly connected components.
Compute all strongly connected components in the graph. Report them in alphabetic order, and within each component, indicates vertices in alphabetic order (i.e., ACZ and not CAZ or ZAC). Separate components with a single



space. No other symbols must be included in the response. The following is an example of the response format: ABC EF XY etc.

Exercise 7 (1.5 points)
Given the following undirected and weighted graph, find a minimum spanning tree using Prim's algorithm. Start from vertex A.



Indicate the total weight of the final minimum spanning tree. Report one single integer value. No other symbols must be included in the response. The following is an example of the response format: 13

Exercise 8 (2.0 points)
Analyze the following program and indicate the exact output it generates.

```
#define R 4
#define C 5
int main(void) {
  int mat1[R][C] ={{ 3,  6, 1, 0, 9},
                   {12,  5, 9, 7, 2},
                   {13,  5, 9, 6, 2},
                   { 1, 14, 6, 9, 5}};
  int mat2[R][C];
  int i, j, ii, jj;
  for (i=0; i<R; i++) {
    for (j=0; j<C; j++) {
      mat2[i][j] = 0;
      for (ii=i-1; ii<=i+1; ii++) {
        for (jj=j-1; jj<=j+1; jj++) {
          if ((ii!=i || jj!=j) && ii>=0 && ii<R && jj>=0 && jj<C) {
            mat2[i][j] += mat1[ii][jj];
          }
        }
      }
      printf ("%d ", mat2[i][j]);
    }
    printf ("\n");
  }
  return 0;
}
```

Reports the value stored in `mat2[0][0]`, `mat2[1][3]`, and `mat2[2][4]`, in this order.

Exercise 9 (2.0 points)

The following function visits the graph `g`.

```
void visit (graph_t *g, vertex_t *n) {
  queue_t *qp;
  vertex_t *d;
  edge_t *e;
  qp = queue_init (g->nv);
  n->color = WHITE;   // LINE 1
  n->dist = 0;
  n->pred = NULL;
  queue_put (qp, (void *)n);
  while (!queue_empty_m(qp)) {
    queue_get(qp, (void **)&n);
    e = n->head;
    while (e != NULL) {
      d = e->dst;
      if (d->color == WHITE) {
        d->color = GREY;
        d->dist = n->dist;       // LINE 2
        d->pred = n->pred + 1;   // LINE 3
        queue_put (qp, (void *)d);
      }
      e = e->next;
    }
    n->color = BLACK;
  }
  queue_dispose (qp, NULL);
}
```

Indicate which one of the following statements is correct. Note that incorrect answers imply a penalty in the final score.

1. It implements a Depth-First Search (DFS).
2. It has a time cost proportional to $|V| + |E|$.
3. Line 1, must be `n->color = BLACK`.
4. Line 1, must be `n->color = GREY`.
5. Line 2, must be `n->dist = n->dist + 1`.
6. Line 3, must be `d->pred = n`.
7. A DFS may visit more nodes than a BFS.
8. A DFS is generally more efficient than a BFS because it is recursive and uses a stack to store gray nodes instead of a queue.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the precise program output with no extra symbols.

```
void f1 (char *);
void f2 (char *);
void f3 (char *);
int main (void){
  char *s = "0b,1C,2d,3E,4f,5H";
  char *p;
  p = s;
  f1 (p);
  return 0;
}
void f1 (char *p) {
  if (*p=='\0')
    return;
  if (*p>='0' && *p<='9') {
    printf ("%c", *p);
  }
  f2 (p+1);
  return;
}
void f2 (char *p) {
  if (*p=='\0')
    return;
  if (*p>='A' && *p<='Z') {
    printf ("%c", *p);
  }
  f3 (p+1);
  return;
}
void f3 (char *p) {
  if (*p=='\0')
    return;
  if (*p=='.' || *p==',' || *p==':' || *p==';') {
    printf ("%c", *p);
  }
```

```
  f1 (p+1);
  return;
}
```

Exercise 11 (3.0 points)
Write the function

```
char *reorder_string (char *s1, int *v);
```

which receives a string s1 and an array of integer v as parameters. The function must allocate a new string `s2` of the same length as `s1` and copy all characters from `s1` into `s2` moving each character into the position specified by the array `v`.
For example, if `s1 = ''abcdefgh''` and `v = {7, 4, 3, 0, 1, 2, 5, 6}`, function `reorder_string` must generate `s2 = ''defcbgha''` and return its pointer. The function must also verify the correctness of the array `v` as far as values out of bound are concerned. For example, the values 8 (or -2) should not be included in `v` as they do not indicate correct characters within string `s1`. In this case, the function should generate a warning and return a NULL string. All other possible errors (e.g., duplicated or absent values must not be verified).
Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

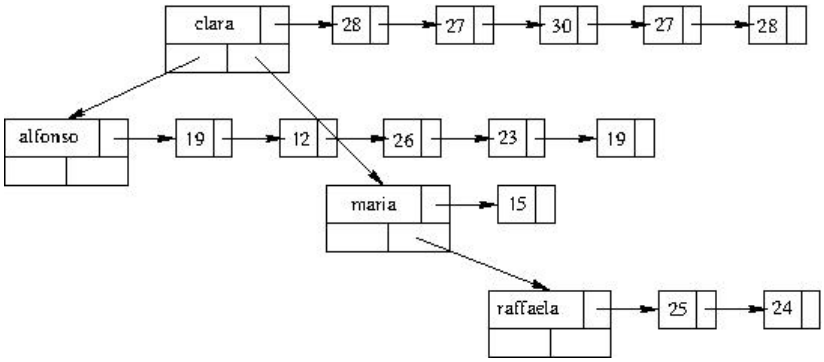
Exercise 12 (5.0 points)
A file includes an undefined number of rows. On each row the first field is a string (of maximum of 100 characters), and the second an integer value that indicates the number of integer values following it on the same file line. For example, the following is a correct file with this format:

```
clara 2 28 27
alfonso 3 19 23 26
maria 1 15
clara 3 30 27 28
raffaela 2 24 25
alfonso 2 12 19
```

Write the function:

```
bst_t *file_to_bst_of_lists (char *name);
```

which receives the file name as a parameter and returns the pointer to a BST in which each node points to the list of integer values associated with the string. The rules for storing the file into a BST of lists are as follows. The BST has the strings as keys (and keys are not duplicated). If a string appears more than once in the file (e.g., `alfonso`, in the previous example), the corresponding numbers must be concatenated in the same secondary list. All secondary lists store the values associated with the corresponding strings. Those values may appear in any order and can be duplicated.
The following picture represents an example of the BST of lists corresponding to the previos file.



Thus, for example, the list related to the BST node with key `alfonso` must include the value $19, 23, 26, 12, 19$ in any order. Define the type `bst_t` and `list_t` for the BST and the list. The string within the BST must be dynamically allocated.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 13 (9.0 points)
A set of bulbs are turned on and off by a set of switches. Initially, all bulbs are turned off. Each switch is connected to some bulbs; pressing the switch will turn each of them on if they are off, and it will turn each of them off if they are on. An electrician would like to understand which is the smaller set of switches (if it exists) that have to be pressed to turn on all bulbs.
A matrix stores the "electrical connections", i.e., the bulbs that can be turned on and off by each switch.
For example, with 4 switches (rows) and 5 bulbs (columns), the matrix:

	0	1	2	3	4
0	1	1	0	0	1
1	1	0	1	0	0
2	0	1	1	1	0
3	1	0	0	1	0

indicates that the first switch (switch number 0) turns on and off bulbs 0, 1, and 4.
Write the function

```
void switches (int **mat, int switch, int bulb);
```

which receives the matrix represented above, with the number of switches (`switch`), the number of bulbs (`bulb`), and it writes (on standard output) the solution to the problem (i.e., the switches to press) if one solution exists.
In the above case, the minimum number of switches able to turn on all the bulbs is 3 with switches $\{0, 1, 3\}$.
Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.